

服务器推送技术

1. 主流服务器推送技术

1.1. 背景

1.2. 服务端推送常用技术

1.2.1. 全双工通信：WebSocket

1.2.2. 服务端主动推送：SSE (Server Send Event)

1.2.3. WebSocket 与 SSE 比较

2. 服务端推送事件 SSE 练习

3. 双工通信 WebSocket

4. WebSocket 通信练习

1. 主流服务器推送技术

1.1. 背景

若干年前，所有的请求都是由浏览器端发起，浏览器本身并没有接受请求的能力。所以一些特殊需求都是用 Ajax 轮询的方式来实现的。比如：

- 股价展示页面实时的获取股价更新；
- 赛事的文字直播，实时更新赛况；
- 通过页面启动一个任务，前端想知道任务后台的实时运行状态。

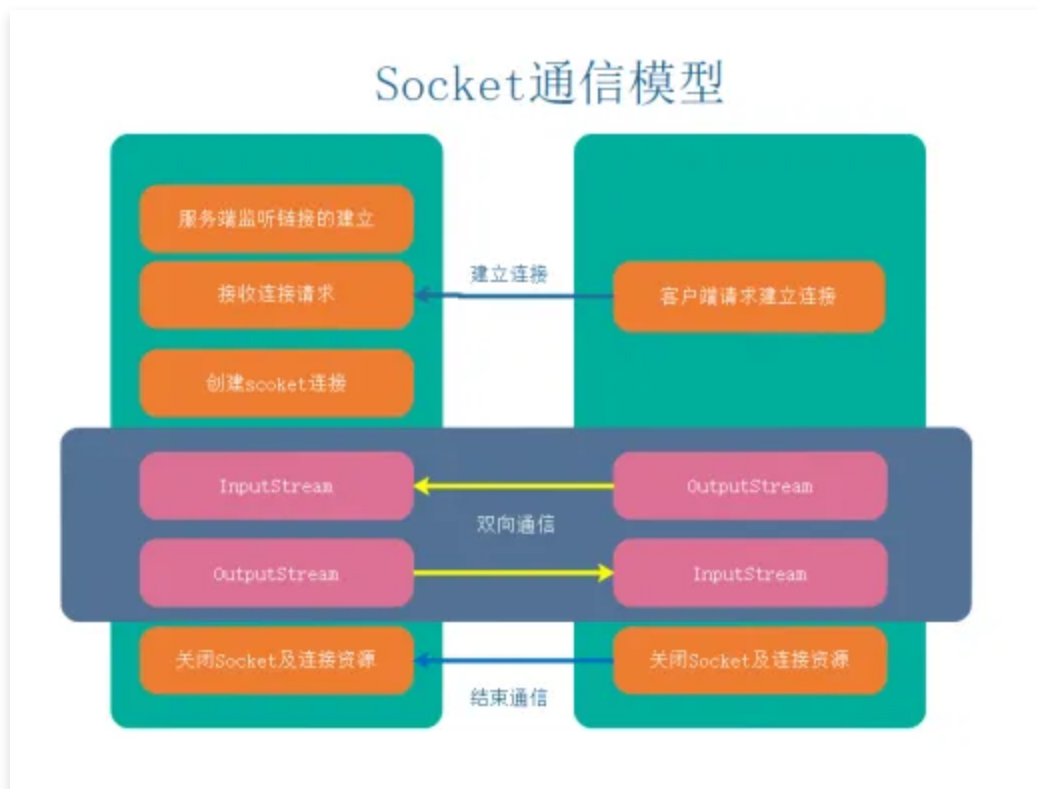
通常的做法就是需要以较小的间隔，频繁的向服务器建立 HTTP 连接询问任务状态的更新，然后刷新页面显示状态。但这样做的后果就是浪费大量流量，对服务端造成了非常大的压力。

1.2. 服务端推送常用技术

在 HTML5 被广泛推广之后，我们就可以使用**服务端主动推送数据**，**浏览器接收数据**的方式来解决上面提到的问题。下面就介绍两种服务端数据推送技术。

1.2.1. 全双工通信：WebSocket

全双工就是双向通信。如果说 HTTP 协议是“对讲机”之间的通话（你一句我一句，有来有回），那 WebSocket 就是移动电话(可以随时发送信息与接收信息，就是全双工)。



其本质上是一个额外的 TCP 连接，建立和关闭时握手使用 HTTP 协议，其他数据传输不使用 HTTP 协议，更加复杂一些，比较适用于需要进行复杂双向实时数据通讯的场景。在 Web 网页上面的客服、聊天室一般都是使用 WebSocket 协议来开发的。

1.2.2. 服务端主动推送：SSE (Server Send Event)

服务器推送事件（Server-Sent Events, SSE）是一种用于实现服务器向客户端单向推送数据的技术。它允许服务器在任何时候向客户端发送数据，而无需客户端明确地请求数据。SSE 基于 HTTP 协议，使用简单的文本格式发送数据，与传统的 AJAX 长轮询或 WebSocket 相比，SSE 更简单易用，适用于需要从服务器获取实时数据更新的应用场景。

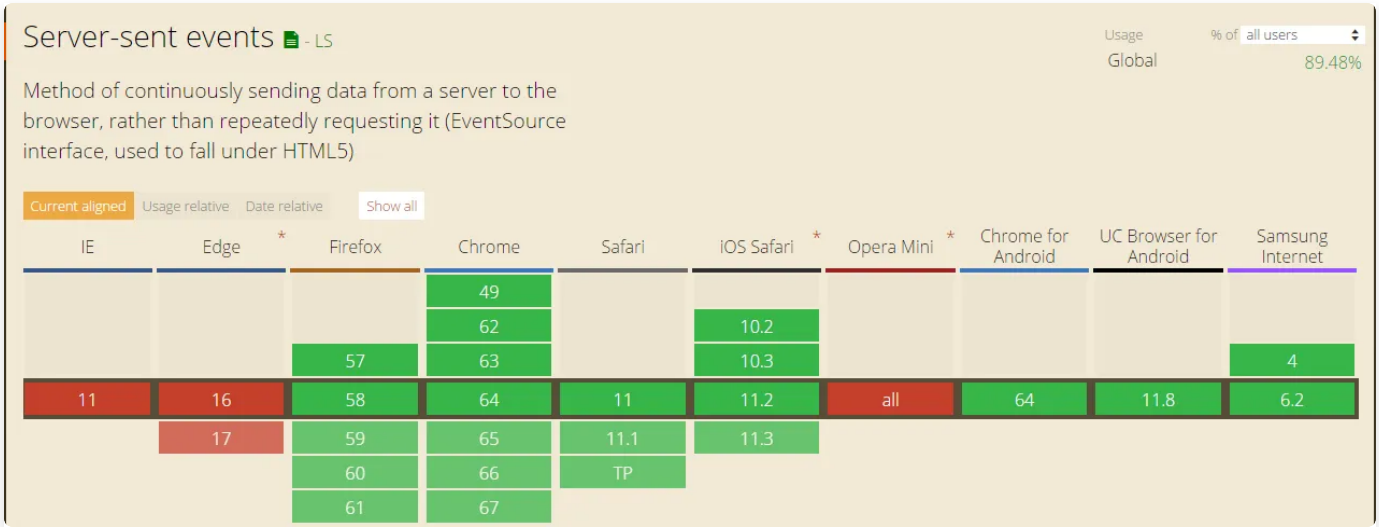
SSE 的基本工作原理是，客户端通过简单的 HTTP 连接向服务器发送一个请求，而服务器在有新数据可供发送时，使用该连接将数据发送给客户端。连接保持打开状态，直到客户端显式地关闭连接或者服务器发生错误。因此，SSE 是一种轻量级的、适合实时通信的解决方案，特别适用于需要实时更新的应用程序，如股票报价、天气预报、社交媒体通知等。

SSE 的一大特色就是重复利用 1 个连接来接收服务端发送的消息（又称 event），从而避免不断轮询请求建立连接，造成服务资源紧张。

1.2.3. WebSocket 与 SSE 比较

	是否基于新协议	是否双向通信	是否支持跨域
SSE	否（HTTP）	否（单向）	否（Firefox 支持跨域）
WebSocket	是（ws）	是	是

但是 IE 和 Edge 浏览器不支持 SSE，所以 SSE 目前的应用场景比较少。虽然 WebSocket 在很多比较旧的版本浏览器上面也不兼容，但是总体上比 SSE 要好不少。另外还有一些开源的 JS 前端产品，如 [SockJS](#)，[Socket.IO](#)，在浏览器端提供了更好的 WebSocket 前端编程体验，与浏览器有更好的兼容性。



2. 服务端推送事件 SSE 练习

新建 spring-boot-websocket 模块，自行创建好包结构和启动主类。

新建 controller 子包，新建服务端推送接口类 SseController，代码参考如下。

```
1 package top.mqxu.springboot.websocket.controller;
2
3 import lombok.extern.slf4j.Slf4j;
4 import org.springframework.http.MediaType;
5 import org.springframework.stereotype.Controller;
6 import org.springframework.web.bind.annotation.RequestMapping;
7 import org.springframework.web.bind.annotation.RequestMethod;
8 import org.springframework.web.servlet.mvc.method.annotation.ResponseBodyE
9 mitter;
10 import org.springframework.web.servlet.mvc.method.annotation.SseEmitter;
11
12 import java.io.IOException;
13 import java.text.DecimalFormat;
14 import java.util.ArrayList;
15 import java.util.List;
16
17 @Slf4j
18 @Controller
19 public class SseController {
20
21     @RequestMapping(value = "/server/info", method = {RequestMethod.GET},
22 produces = "text/event-stream;charset=UTF-8")
23     public ResponseBodyEmitter pushMsg() throws IOException {
24         final SseEmitter emitter = new SseEmitter(0L);
25         log.info("emitter push message .....");
26         List<String> list = new ArrayList<>();
27         list.add("aaa");
28         list.add("bbb");
29         list.add("ccc");
30         emitter.send(list.toString(), MediaType.TEXT_EVENT_STREAM);
31         return emitter;
32     }
33
34     @RequestMapping(value = "/server/data", method = {RequestMethod.GET},
35 produces = "text/event-stream;charset=UTF-8")
36     public ResponseBodyEmitter push() throws InterruptedException, IOExcep
37 tion {
38         final SseEmitter emitter = new SseEmitter(0L);
39         Thread.sleep(1000);
40         double money = Math.random() * 10;
41         DecimalFormat df = new DecimalFormat(".00");
42         String param = df.format(money);
43         emitter.send("白菜价格行情:" + param + "元" + "\n\n", MediaType.TEXT
44 _EVENT_STREAM);
```

```

41         return emitter;
42     }
43
44 }

```

对于服务器端向浏览器发送的数据，浏览器端需要在 JavaScript 中使用 EventSource 对象来进行处理。EventSource 使用的是标准的事件监听器方式，只需要在对象上添加相应的事件处理方法即可。EventSource 提供了三个标准事件。

事件名称	事件触发说明	事件处理方法
open	当服务器向浏览器第一次发送数据时产生	onopen
message	当收到服务器发送的消息时产生	onmessage
error	当出现异常时产生	onerror

除了使用标准的事件处理方法，还可以使用 **addEventListener** 方法对事件进行监听。

```

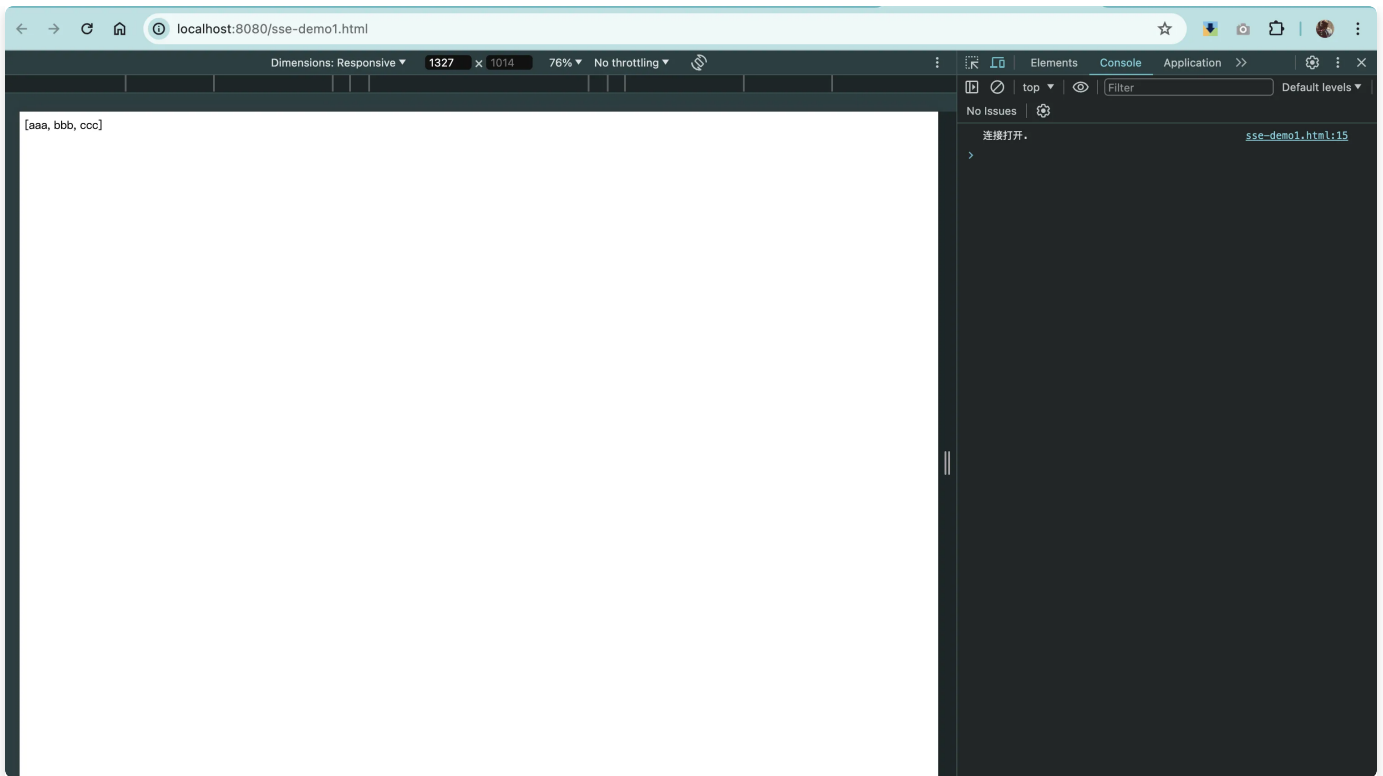
JavaScript
1  var es = new EventSource('事件源名称'); //与事件源建立连接
2  //标准事件处理方法,还有onopen、onerror
3  es.onmessage = function(e) {
4  };
5  //可以监听自定义的事件名称
6  es.addEventListener('自定义事件名称', function(e) {
7  });

```

新建 resources/public/sse-demo1.html，使用原生 JS 来调用前面编写的 SSE 接口 /server/info，参考代码如下。

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>sse-demo1</title>
6 </head>
7 <body>
8   <div id="msg_from_server">
9 </div>
10 <script>
11   if (!!window.EventSource) {
12     let source = new EventSource('http://localhost:8080/server/info');
13     let s = '';
14     source.addEventListener('open', () => {
15       console.log("连接打开.");
16     }, false);
17     source.addEventListener('message', (e) => {
18       s += e.data + "<br/>"
19       document.getElementById("msg_from_server").innerHTML = s;
20     });
21     source.addEventListener('error', (e) => {
22       if (e.readyState === EventSource.CLOSED) {
23         console.log("连接关闭");
24       } else {
25         console.log(e.readyState);
26       }
27     }, false);
28   } else {
29     alert(4);
30     console.log("没有sse");
31   }
32 </script>
33 </body>
34 </html>
```

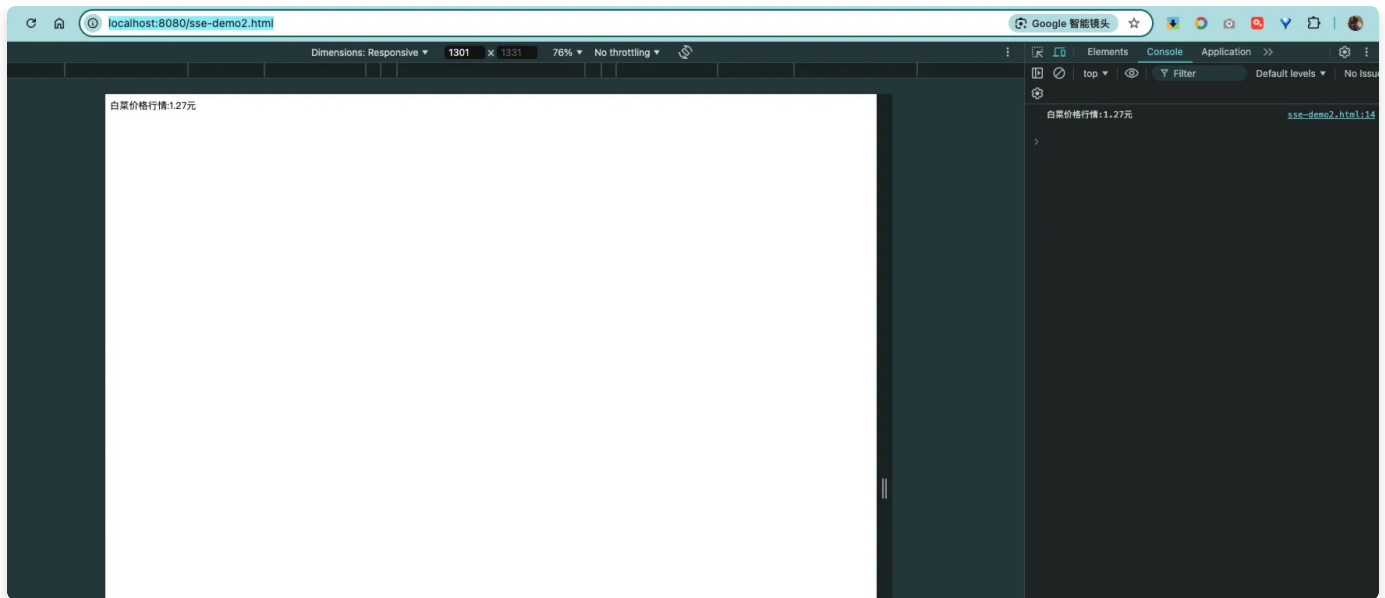
启动应用，打开 <http://localhost:8080/sse-demo1.html>，运行效果如下。



再新建 resources/public/sse-demo2.html，调用 /server/data 接口，参考代码如下。

```
HTML |
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <title>sse-demo2</title>
6  </head>
7  <body>
8  <div id="result">
9  </div>
10 <script>
11   //需要判断浏览器支不支持，可以去w3c进行查看
12   const source = new EventSource('http://localhost:8080/server/data');
13   source.onmessage = function (event) {
14     console.info(event.data);
15     document.getElementById('result').innerText = event.data
16   };
17 </script>
18 </body>
19 </html>
```

<http://localhost:8080/sse-demo2.html>，运行效果：1 秒后出现服务端推送的数据。



3. 双工通信 WebSocket

WebSocket 是一种在单个 TCP 连接上提供全双工通信的网络协议。它允许在客户端和服务端之间进行双向通信，这意味着服务器可以主动向客户端推送数据，而不需要客户端发起请求。

与传统的 HTTP 请求-响应模型不同，WebSocket 连接通过一个握手过程建立，并在建立连接后保持打开状态。一旦连接建立，客户端和服务端之间就可以直接发送消息，而无需每次通信都发起新的 HTTP 请求。这减少了通信开销并提高了实时性能。

WebSocket 的应用场景非常广泛，主要因为它能够实现实时、高效的双向通信。以下是一些常见的 WebSocket 应用场景：

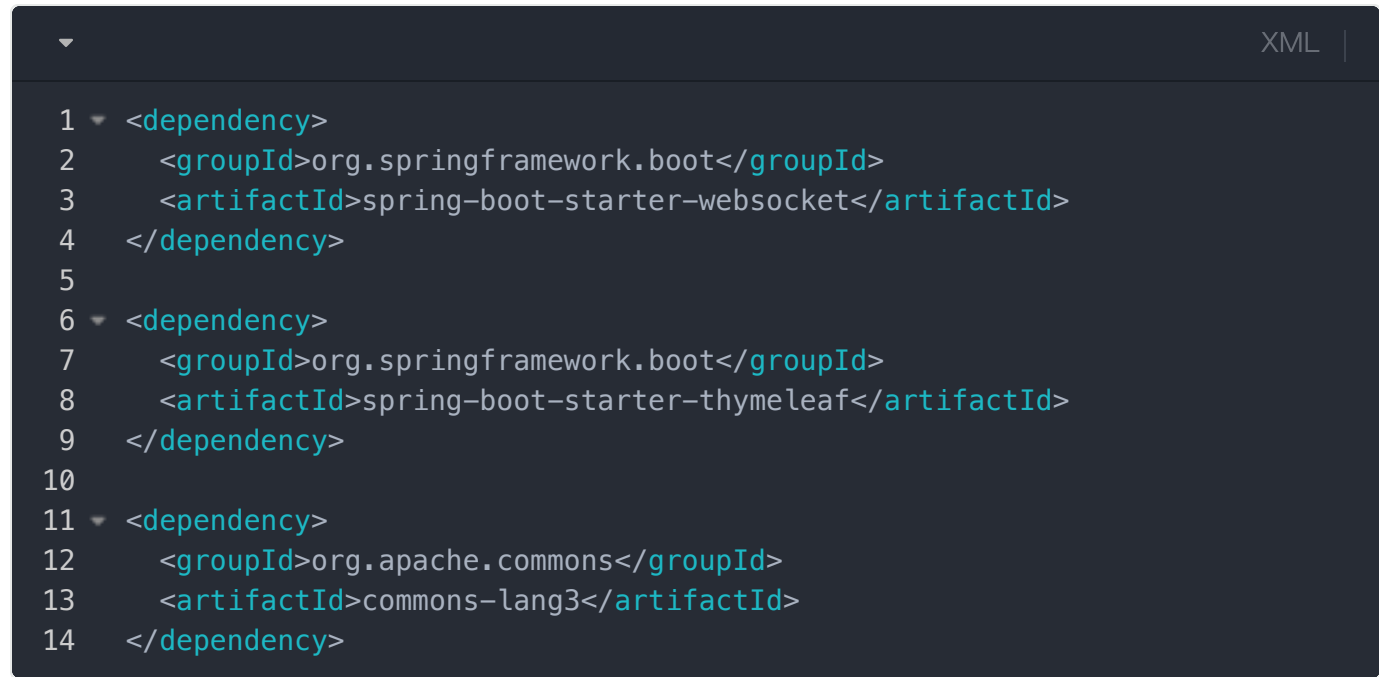
- 实时聊天应用程序：** WebSocket 最常见的用途之一是实时聊天应用程序，如即时通讯工具、社交媒体平台、在线客服系统等。用户可以实时发送和接收消息，而不需要手动刷新页面或者发起额外的请求。
- 实时协作工具：** WebSocket 使得实时协作工具（如在线文档编辑器、协作白板、团队任务管理工具等）能够在多个用户之间同步数据，并实时反映其他用户的操作。
- 实时游戏：** 多人在线游戏和其他需要实时交互的游戏，如棋类游戏、卡牌游戏等，可以使用 WebSocket 来实现玩家之间的实时通信和游戏状态同步。
- 股票市场和金融交易：** WebSocket 在金融领域的应用十分广泛，可以用于实时股票报价、交易执行通知、财务数据更新等方面。

5. **实时数据监控和更新：** 各种实时数据监控系统（如网络监控、服务器监控、物联网设备监控等）可以使用 WebSocket 来实时推送数据更新和警报通知。
6. **在线视频和音频流：** WebSocket 可以用于实时传输视频和音频流，例如视频会议、音视频直播等应用场景。

总的来说，任何需要实时、双向通信的 Web 应用场景都可以考虑使用 WebSocket 来实现。它提供了一种高效、低延迟的通信方式，可以大大提升用户体验和应用程序的功能性。

4. WebSocket 通信练习

首先添加依赖。

A screenshot of a code editor with a dark theme. The editor shows an XML configuration for dependencies. The text is as follows:

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-websocket</artifactId>
4 </dependency>
5
6 <dependency>
7   <groupId>org.springframework.boot</groupId>
8   <artifactId>spring-boot-starter-thymeleaf</artifactId>
9 </dependency>
10
11 <dependency>
12   <groupId>org.apache.commons</groupId>
13   <artifactId>commons-lang3</artifactId>
14 </dependency>
```

The editor has a tab labeled 'XML' in the top right corner. Line numbers 1 through 14 are visible on the left side of the code block.

config 子包新建 WebSocketConfig 配置类，如下。

```
1 package top.mqxu.boot.websocket.config;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.web.socket.config.annotation.EnableWebSocket;
6 import org.springframework.web.socket.server.standard.ServerEndpointExporter;
7
8 @Configuration
9 @EnableWebSocket
10 public class WebSocketConfig {
11     /**
12      * 注入ServerEndpointExporter,
13      * 这个bean会自动注册使用了@ServerEndpoint注解声明的WebSocket Endpoint
14      */
15     @Bean
16     public ServerEndpointExporter serverEndpointExporter() {
17         return new ServerEndpointExporter();
18     }
19 }
```

定义 WebSocket 服务器：server 子包新建 WebSocketServer 类，如下。

```
1  import com.fasterxml.jackson.core.type.TypeReference;
2  import com.fasterxml.jackson.databind.ObjectMapper;
3  import io.micrometer.common.util.StringUtils;
4  import jakarta.websocket.*;
5  import jakarta.websocket.server.PathParam;
6  import jakarta.websocket.server.ServerEndpoint;
7  import lombok.extern.slf4j.Slf4j;
8  import org.springframework.stereotype.Component;
9
10 import java.io.IOException;
11 import java.util.Map;
12 import java.util.concurrent.ConcurrentHashMap;
13
14
15 @Component
16 @ServerEndpoint("/server/{uid}")
17 @Slf4j
18 public class WebSocketServer {
19     /**
20      * 记录当前在线连接数
21      */
22     private static int onlineCount = 0;
23
24     /**
25      * 使用线程安全的ConcurrentHashMap来存放每个客户端对应的WebSocket对象
26      */
27     private static final ConcurrentHashMap<String, WebSocketServer> WEB_S
OCKET_MAP = new ConcurrentHashMap<>();
28
29     /**
30      * 与某个客户端的连接会话，需要通过它来给客户端发送数据
31      */
32     private Session session;
33
34     /**
35      * 接收客户端消息的uid
36      */
37     private String uid = "";
38
39     /**
40      * 连接建立成功调用的方法
41      *
42      * @param session session
43      * @param uid      用户id
44      */
45 }
```

```

45 @OnOpen
46 public void onOpen(Session session, @PathParam("uid") String uid) {
47     this.session = session;
48     this.uid = uid;
49     if (WEB_SOCKET_MAP.containsKey(uid)) {
50         WEB_SOCKET_MAP.remove(uid);
51         //加入到set中
52         WEB_SOCKET_MAP.put(uid, this);
53     } else {
54         //加入set中
55         WEB_SOCKET_MAP.put(uid, this);
56         //在线数加1
57         addOnlineCount();
58     }
59
60     log.info("用户{}连接成功, 当前在线人数为:{}", uid, getOnlineCount());
61     try {
62         sendMsg("连接成功");
63     } catch (IOException e) {
64         log.error("用户{}网络异常!", uid, e);
65     }
66 }
67
68 /**
69  * 连接关闭调用的方法
70  */
71 @OnClose
72 public void onClose() {
73     if (WEB_SOCKET_MAP.containsKey(uid)) {
74         WEB_SOCKET_MAP.remove(uid);
75         //从set中删除
76         subOnlineCount();
77     }
78     log.info("用户{}退出, 当前在线人数为:{}", uid, getOnlineCount());
79 }
80
81 /**
82  * 收到客户端消息后调用的方法
83  *
84  * @param message 客户端发送过来的消息
85  * @param session 会话
86  */
87 @OnMessage
88 public void onMessage(String message, Session session) {
89     log.info("用户{}发送报文:{}", uid, message);
90     //群发消息
91     if (StringUtils.isNotBlank(message)) {
92         try {

```

```

93         //解析发送的报文
94         ObjectMapper objectMapper = new ObjectMapper();
95         Map<String, String> map = objectMapper.readValue(message,
96             new TypeReference<>() {
97             });
98         //追加发送人(防止串改)
99         map.put("fromUID", this.uid);
100         String toUID = map.get("toUID");
101         //传递给对应的toUserId用户的WebSocket
102         if (StringUtils.isNotBlank(toUID) && WEB_SOCKET_MAP.containsKey(toUID)) {
103             WEB_SOCKET_MAP.get(toUID).sendMsg(objectMapper.writeValueAsString(map));
104         } else {
105             //若果不在这个服务器上, 可以考虑发送到mysql或者redis
106             log.error("请求目标用户{}不在该服务器上", toUID);
107         }
108     } catch (Exception e) {
109         log.error("用户{}发送消息异常! ", uid, e);
110     }
111 }
112
113 /**
114  * 处理错误
115  *
116  * @param session session
117  * @param error    error
118  */
119 @OnError
120 public void onError(Session session, Throwable error) {
121     log.error("用户{}处理消息错误, 原因:{}", this.uid, error.getMessage());
122     error.printStackTrace();
123 }
124
125 /**
126  * 实现服务器主动推送
127  *
128  * @param msg 消息
129  */
130 private void sendMsg(String msg) throws IOException {
131     this.session.getBasicRemote().sendText(msg);
132 }
133
134
135 /**
136  * 发送自定义消息

```

```

137      *
138      * @param message 消息
139      * @param uid      用户id
140      */
141      public static void sendInfo(String message, @PathParam("uid") String
142      uid) throws IOException {
143          log.info("发送消息到用户{}发送的报文:{}", uid, message);
144          if (!StringUtils.isEmpty(uid) && WEB_SOCKET_MAP.containsKey(uid))
145          {
146              WEB_SOCKET_MAP.get(uid).sendMsg(message);
147          } else {
148              log.error("用户{}不在线! ", uid);
149          }
150      }
151      private static synchronized int getOnlineCount() {
152          return onlineCount;
153      }
154      private static synchronized void addOnlineCount() {
155          WebSocketServer.onlineCount++;
156      }
157      private static synchronized void subOnlineCount() {
158          WebSocketServer.onlineCount--;
159      }
160      }
161  }

```

resources/public 目录, 新建 websocket.html, 如下。

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="utf-8">
5   <title>WebSocket消息通知</title>
6 </head>
7 <body>
8 <div>
9   <label for="uid">【uid】 : </label>
10  <input id="uid" name="uid" type="text" value="1">
11 </div>
12
13 <div>
14   <label for="toUID">【toUID】 : </label>
15   <input id="toUID" name="toUID" type="text" value="2">
16 </div>
17
18 <div>
19   <label for="msg">【Msg】 : </label>
20   <input id="msg" name="msg" type="text" value="hello WebSocket">
21 </div>
22
23 <div>
24   【第一步操作:】 :
25   <button onclick="openSocket()">开启socket</button>
26 </div>
27
28 <div>
29   【第二步操作:】 :
30   <button onclick="sendMessage()">发送消息</button>
31 </div>
32
33 <script>
34   let socket;
35
36   //打开WebSocket
37   function openSocket() {
38     if (typeof (WebSocket) === "undefined") {
39       console.log("您的浏览器不支持WebSocket");
40     } else {
41       console.log("您的浏览器支持WebSocket");
42       //实现化WebSocket对象, 指定要连接的服务器地址与端口, 建立连接。
43       let socketUrl = "http://localhost:8080/server/" + document.getElementById("uid").value;
44       //将https与http协议替换为ws协议
```

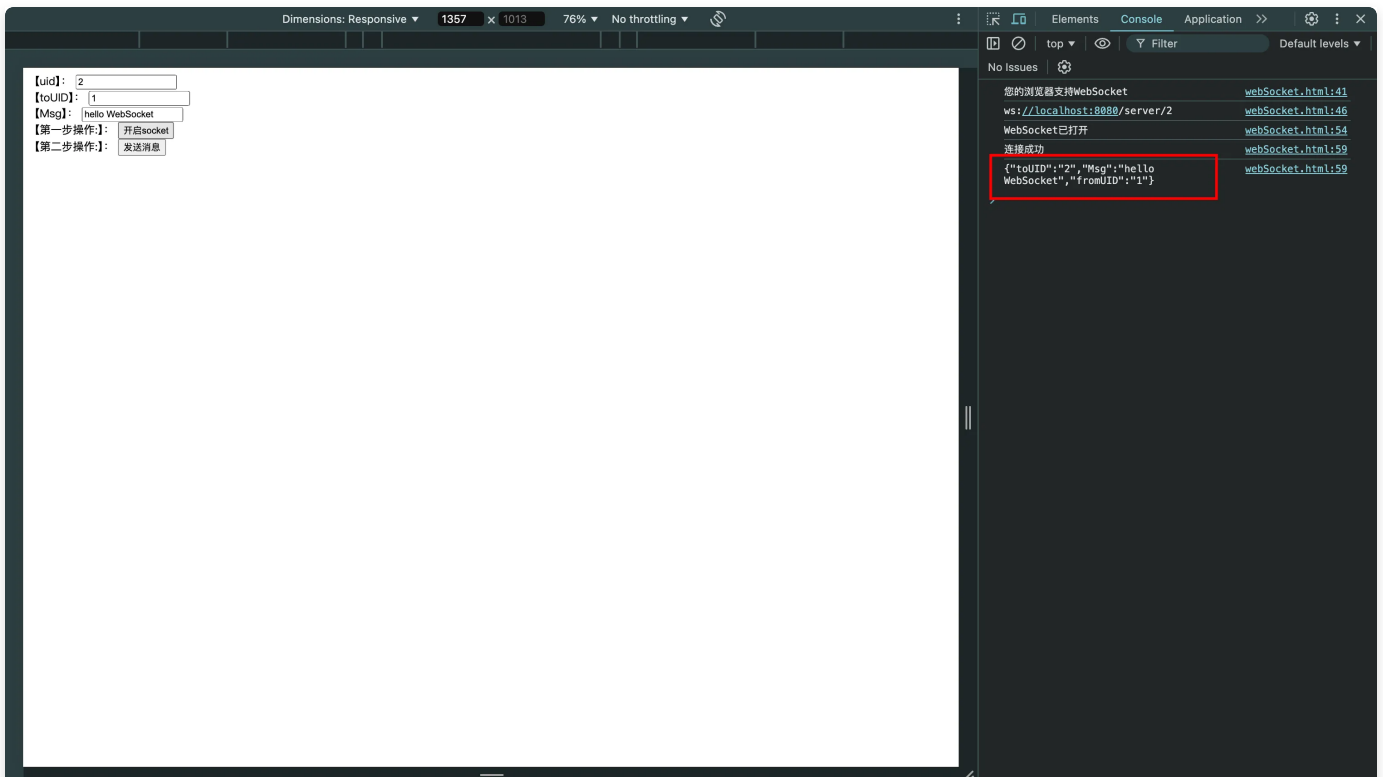
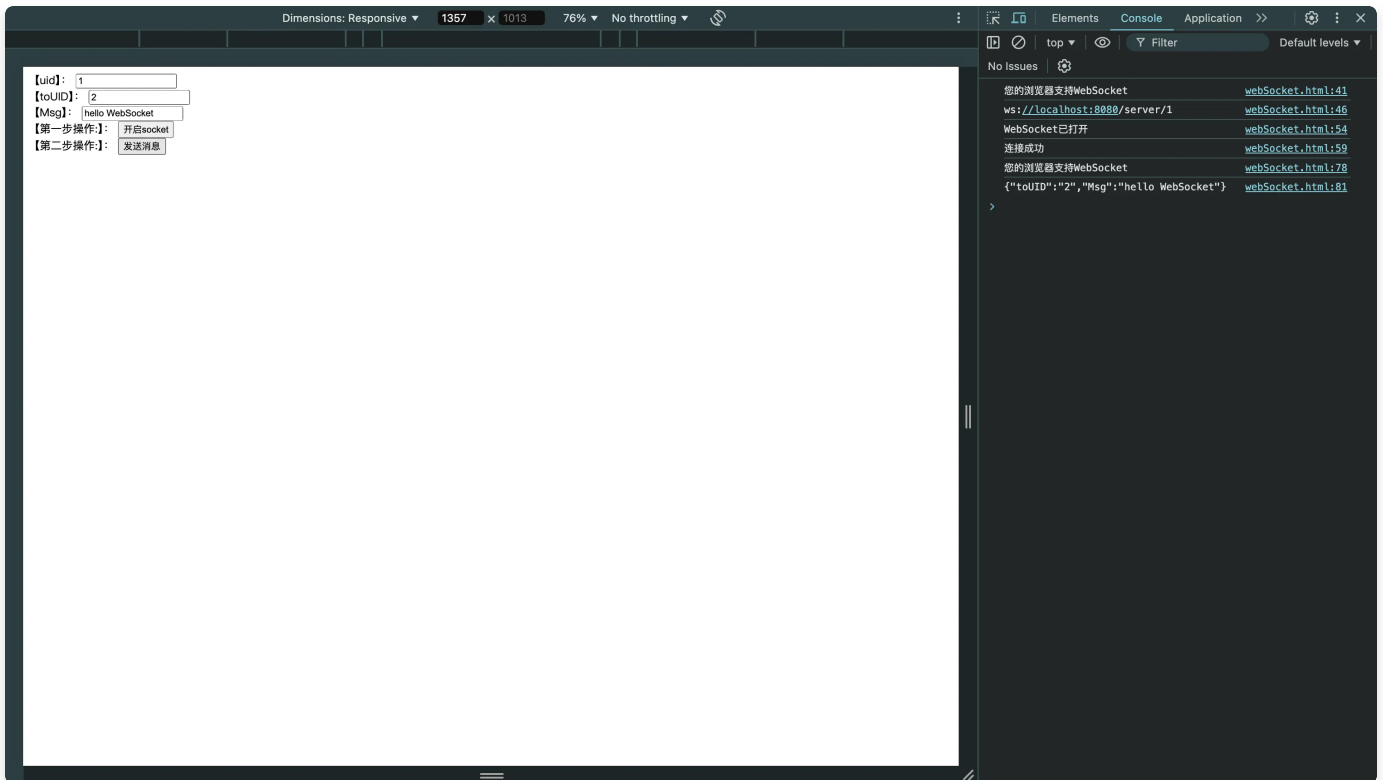
```

45         socketUrl = socketUrl.replace("https", "ws").replace("http",
46         "ws");
47         console.log(socketUrl);
48         if (socket !== null) {
49             socket.close();
50             socket = null;
51         }
52         socket = new WebSocket(socketUrl);
53         //打开事件
54         socket.onopen = function () {
55             console.log("WebSocket已打开");
56             //socket.send("这是来自客户端的消息" + location.href + new Date());
57         };
58         //获得消息事件
59         socket.onmessage = function (msg) {
60             console.log(msg.data);
61             //发现消息进入,开始处理前端触发逻辑
62         };
63         //关闭事件
64         socket.onclose = function () {
65             console.log("WebSocket已关闭");
66         };
67         //发生了错误事件
68         socket.onerror = function () {
69             console.log("WebSocket发生了错误");
70         }
71     }
72 }
73
74 //发送消息
75 function sendMessage() {
76     if (typeof (WebSocket) === "undefined") {
77         console.log("您的浏览器不支持WebSocket");
78     } else {
79         console.log("您的浏览器支持WebSocket");
80         const toUID = document.getElementById("toUID").value
81         const msg = document.getElementById("msg").value
82         console.log('{"toUID":"' + toUID + '","Msg":"' + msg + '"}');
83         socket.send('{"toUID":"' + toUID + '","Msg":"' + msg + '"}');
84     }
85 }
86 </script>
87 </body>
88 </html>

```


启动应用，打开两个浏览器，都访问 <http://localhost:8080/webSocket.html>，分别模拟用户 1 向用户 2 发信息、用户 2 向用户 1 发信息。

分别先点击两个页面的“开启 socket”，然后再输入消息发送，查看控制台，如下。



IDEA 控制台信息如下。

```
INFO 92679 --- [nio-8080-exec-1] t.m.b.websocket.server.WebSocketServer : 用户1连接成功,当前在线人数为:1
INFO 92679 --- [nio-8080-exec-2] t.m.b.websocket.server.WebSocketServer : 用户2连接成功,当前在线人数为:2
INFO 92679 --- [nio-8080-exec-3] t.m.b.websocket.server.WebSocketServer : 用户2发送报文:{"toUID":"1","Msg":"hello from user2"}
INFO 92679 --- [nio-8080-exec-4] t.m.b.websocket.server.WebSocketServer : 用户1发送报文:{"toUID":"2","Msg":"hello WebSocket"}
```